

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 01 • NIVEL 3 - AVANZADO

Clase 01: Fundamentos de LLMs, Tokens y Arquitectura Transformer

«Modelos de Lenguaje como Motores de Predicción Probabilística»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 01** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la tokenización (BPE), ventana de contexto, inferencia probabilística y temperatura.

Competencia Práctica: Calcular tokens, estructurar prompts y configurar hiperparámetros de inferencia.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 01: Fundamentos de LLMs, Tokens y Arquitectura Transformer

Los Modelos de Lenguaje Grande (LLMs) son redes neuronales basadas en la arquitectura Transformer que predicen el siguiente token.

☀ Metáfora Central: Modelos de Lenguaje como Motores de Predicción Probabilística

Un LLM es como el teclado predictivo de tu móvil, pero entrenado con todo el conocimiento digital del planeta.

Principios Teóricos y Modelo Mental

Los LLMs no procesan palabras ni letras, procesan 'tokens' (fragmentos de palabras de ~4 caracteres).

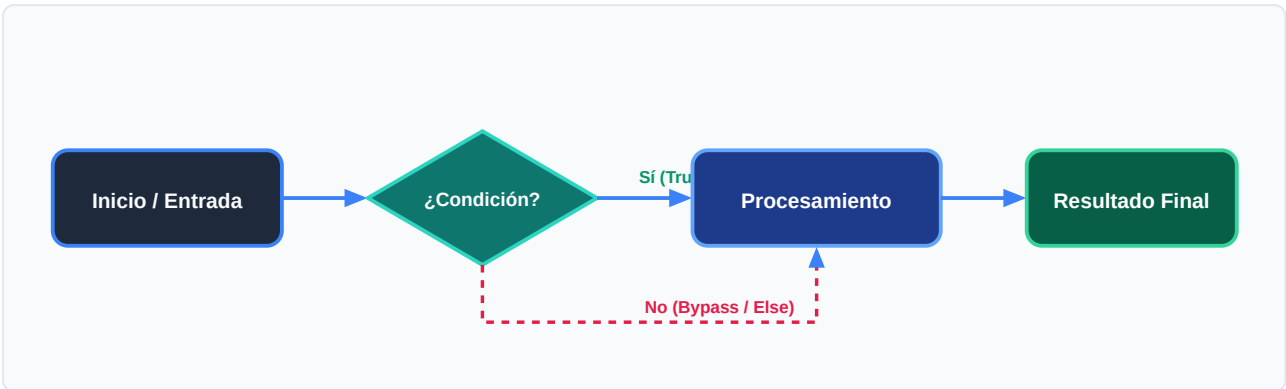
La temperatura controla la entropía de la distribución probabilística (0.0 = determinista, 1.0 = creativo).

⚡ Regla de Oro en Python

Para tareas de extracción estructurada, código o datos, mantén siempre la temperatura en 0.0.

Arquitectura y Movimiento de Datos en Memoria

Ciclo de tokenización, atención Transformer y muestreo de salida.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Conversión de texto a lista de enteros (Token IDs).	Vector de tokens de entrada.
2. Evaluación	Paso hacia adelante en capas de auto-atención.	Distribución de Logits calculada.
3. Transformación	Aplicación de Softmax y Temperatura.	Probabilidades normalizadas.
4. Retorno / Salida	Decodificación de token a texto y streaming.	Respuesta generada token por token.

Visualización Mental

Cada llamada a un LLM es stateless (sin estado); no recuerda nada a menos que le envíes el historial.

Código de Demostración en Python 3.10+

Simulador de tokenización y muestreo probabilístico:

main.py (Python 3.10+)

PEP 8 Compliant

```
def simular_tokenizador(texto: str) -> list[str]:  
    # Simulación básica de subwords  
    return texto.replace(".", " ").split()  
  
tokens = simular_tokenizador("Python es el lenguaje líder en Inteligencia Artificial.")  
print(f"Total tokens: {len(tokens)}")  
print("Tokens extraídos:", tokens)
```

Análisis de Ingeniería del Código

Demostración de descomposición léxica y cálculo de ventana de contexto.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

| Trampas Frecuentes de Depuración (Gotchas)

Gotchas de costos y límites de contexto.

⚠ Gotcha Frecuente (Trampa de Principiante)

Enviar documentos gigantes sin podar agota la ventana de contexto y dispara los costos de tokens.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
prompt = doc_entero_de_500_paginas + '\nResume  
esto' # ❌ Desborda el contexto
```

✅ Patrón Correcto

```
# Chunking previo y filtrado semántico RAG ✅
```



Consejo de Resiliencia en Producción

Monitorea siempre el consumo de `prompt_tokens` y `completion_tokens` en producción.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 01: Fundamentos de LLMs, Tokens y Arquitectura Transformer**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.



Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Modelos de Lenguaje como Motores de Predicción Probabilística
2. Regla de Oro	Para tareas de extracción estructurada, código o datos, mantén siempre la temperatura en 0.0.
3. Gotcha Crítico	Enviar documentos gigantes sin podar agota la ventana de contexto y dispara los costos de tokens.
4. Buenas Prácticas	Monitorea siempre el consumo de <code>prompt_tokens</code> y <code>completion_tokens</code> en producción.



Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una función que estime el costo en USD de una llamada de inferencia dado un número de palabras.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.